

BioSpecInfo

Test Documentation — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v181** · commit `53da012`

Generato il 2026-10-02

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Test Documentation — BioSpecInfo

Field	Value
Software	BioSpecInfo
Version described	bsi-v181
Purpose	Describe how the tests are organised, how to run them, what they cover and where they leave gaps.

1. Kinds of test present

Kind	Present	Notes
End-to-end (E2E)	✓	Playwright on headless Chromium against a local HTTP server. It is the prevailing form.
Data verification	✓	Scientific data compared against an independent source
Stability tests	✓	Long sessions, exhausted storage, degraded network
Documentation/code consistency	✓	Verifies that what the documentation promises really exists
Isolated unit tests	✗	See §6
Instrumented code coverage	✓	<code>audit_copertura</code> , with Chromium's profiler: no build, no source rewritten. See §6

Why E2E and not unit tests

The application is made of classic scripts operating directly on the DOM, with no modules and no dependency injection: there are no units that could be isolated without a deep rewrite. The defects that actually occurred — a blurred canvas, a button unreachable because its container is collapsed, an AI provider that stops answering — **would not have surfaced** from a unit test anyway: they live in the integration between code, DOM and browser.

The choice is therefore reasoned, not endured. Its limits remain, and are declared in §6.

2. How to run the tests

Requirements

```
npm install          # playwright-core only
python3 -m http.server 8899 &  # RDKit WASM requires an HTTP context
```

Without a server every test fails: opening the files over `file://` prevents the WebAssembly modules from loading.

The whole battery, with a report

```
node tools/genera-evidenza.js
```

It runs every bench, transcribes its full output and produces `docs/evidence/RAPPORTO-VERIFICA.md` with environment, commit and SHA-256 digests. It exits with code `0` only if no bench fails.

```
node tools/genera-evidenza.js --veloce # skips the benches that need a browser
```

The `--veloce` option is useful for a quick check but **is not sufficient for an attestation**: the benches it skips are precisely the ones measuring real behaviour.

A single bench

```
node tools/verifica-farmaci.js          # a stand-alone tool
node tools/verifica-farmaci.js --json   # machine-readable output

cd tools/banchi && node test_spettri.js # one E2E bench
```

The E2E benches live in `tools/banchi/`, versioned like the rest of the repository; the `BSI_BANCHI` variable allows another folder to be pointed at.

It has not always been so, and that is worth saying. Up to version `bsi-v168` the default folder was the working area of the session in which the benches had been written: **32 of the 39 benches were not in the repository**. Whoever cloned it and ran the documented command found seven, and received no warning — because a missing bench was counted but did not make the battery fail, which closed with "CONFORMANT" and zero failures.

Both things have been corrected: the benches are versioned, and **a missing bench makes the outcome NON-CONFORMANT**. A bench that is not there is not a bench that passed.

3. Composition of the battery

48 benches, grouped by what they demonstrate.

3.1 Scientific data

Bench	Verifies	Checks
verifica-farmaci	Structure $\leftrightarrow$ molecular weight, duplicates, recorded deviations	233 entries
test_spettri	Functional-group recognition on reference molecules	36
test_spettri_ui	Grafting of the engine into the app's spectrum-drawing points	16
test_assi	Axis conventions in the SVG spectra	13
test_assi_canvas	Axis conventions in the canvas spectra	6
test_costanti	Physical-constant lookup and refusal of ambiguities	45
audit_dati	Consistency of the tabulated data	29
test_farm_ui	That the Drug Atlas draws every drug it holds in memory, category by category, and that a never-labelled category still appears	11
test_astro	The astrochemistry data: molecular weight $\leftrightarrow$ formula for the 30 interstellar molecules against IUPAC atomic weights, no repeated celestial body, every quiz question with a valid answer among its options	13
test_datasci	The maths the Data Science section displays : R^2 on an exact relation, standardised weights in the true ratio, ROC on hand-computable values, behaviour on pure noise, and that no section is left unreachable	23
test_simmetria	Point groups, normal modes, IR/Raman selection rules	26
test_cheminfo	The cheminformatics engine: Tanimoto and Dice against hand-computable values, Morgan and MACCS fingerprints, Butina clustering, Bemis-Murcko scaffolds, PCA, kernel ridge, scaffold split, null model by label scrambling , grouped cross-validation, CSV and SDF exports, contrast of the nine panels , agreement between lab and engine, fragmentation, matched pairs against known effects , substructure search, SAR table, model comparison in both directions, enrichment (EF and BEDROC) on hand-computable cases, conformal intervals and their refusal on classification	189

A bench that verifies a model must also verify it when the model is supposed to fail. `test_cheminfo` exercises the null model in both directions: on a learnable signal the true R^2 (0.394) beats all eight label-scrambled twins, with a margin of 0.287; on the **same molecules with random labels** the R^2 (−0.199) does **not** beat them. A check that verifies only the first case would pass even if the guard were wired to "true".

A "contrast 0" that did not cover the new surface. `verifica-accessibilita` measures what is visible, and the six panels of the Cheminformatics section stay hidden until the analysis has been run: it had never seen them. `test_cheminfo` now runs the analysis, opens the panels in turn and measures contrast over **460 text elements**, with the WCAG formulas rewritten inside the bench — asking the code under test how good its own contrast is, is not measuring. The bench also demands that more than 250 were measured: if the panels failed to open, the zero would be empty and the check would fail.

3.2 AI agent

Bench	Condition simulated
test_404	The provider has retired the model
test_503	Temporary overload
test_ko · browser_ko	Provider unreachable from the browser
test_firma	Opaque reasoning signatures to be returned
test_tetto	Token ceiling learned from the error
test_attesa · test_attesalunga	Timeouts and imposed waits
test_doppioinvio	Two overlapping submissions
browser_prova	Reachability measurement without a key
test_nucleo · browser_prov	Agentic loop and provider selection
caccia_ai	Malformed answers, truncated stream, 502 with HTML, Stop during the answer, panel closed halfway, hostile text

3.3 Stability

Bench	What it puts to the test
audit_stabilita	89 sections opened 5 times, full storage, corrupted data, bursts of clicks
audit_promesse	Rejected and unhandled promises, with a healthy network and a dead one
audit_quota	localStorage.setItem forced to fail on 10 pages
test_sw	Offline, degraded network, update while working
test_filemanager	Store unavailable, space exhausted, failed writes
test_visore3d	WebGL context consumption while browsing molecules

3.4 Interface

browser_reset · browser_proxy · browser_proxyui · browser_rdkit · browser_lab · browser_frontiera · test_aggiorna · test_guidaproxy · test_fluidita

test_fluidita — 15 checks. It measures how long the page stays *blocked* at every section switch, across all 89. It found three defects no other bench could see: the "3D PRO Viewer" froze the page for **797 ms** on the first click — the profiler attributes them to 3Dmol's first `render()`, which compiles the WebGL context's shaders; the **296 figures** in "Synthesis" (5,351 SVG nodes) were all drawn before anything was shown, while three are visible on opening; and **296 timers 40 ms apart** were scheduled when that section opened — eleven and a half seconds of wake-ups, each calling `getTotalLength()` on every stroke of every figure, including those off screen.

After the fixes: median switch **14 ms**, worst **162 ms**, **no section above 100 ms** (there were two, the worst at 1,166 ms).

Half the checks are the *opposite proofs*, because a page that builds nothing is instantaneous: the WebGL canvas must appear anyway, all 296 figures must exist shortly after, no frame may stay empty, printing must not come out mute, and the search must filter even cards whose figure is not yet born. And the bench counts the sections it traversed: on ten instead of 89 the timings would be excellent and the measurement worthless.

3.5 Security and accessibility

Bench	Verifies
verifica-sicurezza	API keys in tracked files, clear-text passwords, secrets in <code>wrangler.toml</code> , telemetry, scripts from external domains
verifica-accessibilita	WCAG AA contrast, accessible names, field labels, alternative text, heading hierarchy, <code>lang</code> attribute — across 13 pages
audit_mobile	That at 390 px the page does not scroll horizontally, across all 89 sections
audit_rete	That no user data leaves the device : a canary value seeded into 71 stores, the app used across 6 pages and 89 sections, the URL, headers and body of every request inspected
audit_copertura	How many bytes of JavaScript are actually executed while walking the application: 49.89 %, recorded and defended

The obstacle belonged to the tool, not to the problem. For three versions code coverage was declared unmeasurable, with this reason: "instrumenting would require introducing a build, which the application does not have". That was true of `c8` and `istanbul` , which rewrite the source before running it. But **Chromium collects it by itself**, inside the engine, without touching a byte of the file served: `Profiler.startPreciseCoverage` , which Playwright exposes as `page.coverage.startJSCoverage()` . No build was needed: what was needed was noticing that the constraint belonged to the tools chosen.

The first result was 100 % on every file, including 3.6 MB of `index.html` . That was the bench measuring nothing, in its most insidious form: a perfect number. V8 emits, for each function, an outer range with the number of entries, and inside it ranges with `count: 0` for what was not executed. Summing the positive ranges sums the whole file. One does the opposite: start from the total and subtract the union of the empty ranges.

The real number is **49.89 %**, and it holds the same pact as the accessibility debt: recorded, and the bench fails if it falls. An absolute threshold ("80 %") would be an invented figure; "it must not get worse" is a promise that can be kept.

Why a canary and not a list of requests. SEC-02 — "no personal data leaves the device" — was verified by checking the two *mechanisms* of exit: empty telemetry variable, no script from an external domain. That is real evidence but indirect: it says "I cannot see how it would leave", which is not "it did not leave". Looking instead at the list of hosts contacted would be just as weak, because it concludes from absence of evidence.

The canary reverses the burden. An unrepeatable string is written into the user's own data — notes, chats, API keys, settings, progress — then the application is **used**, and every outgoing request is opened and read. If the string appears nowhere, it is because it really did not leave.

```
BSI_PROVA_FUGA=1 node tools/banchi/audit_rete.js # must FAIL
```

The bench carries its own proof of working: with that variable it causes a leak on purpose, towards a **declared** host — contacting it is legitimate, sending it the user's data is not. If it does not fail then, the instrument is the thing that is broken.

Why a bench on horizontal overflow. UI-03 says "the app must work at 390 px", and the matrix gave it as verified by `audit_stabilita`, which in a phone viewport looks at **JavaScript errors**: a page can have not a single one and still run a hundred and sixty-three pixels off the screen. It was a requirement declared covered and not measured. The bench measures the document's `scrollWidth` — exactly what makes the scrollbar appear — and not any element wider than the screen, which inside a container built to scroll is correct and would fill the output with noise until nobody reads it any more.

What the accessibility check does not cover, and it must be said: text inside SVGs (the colour comes from `fill`, the background is a drawn shape) and text on gradient backgrounds (they do not have a colour). The skipped elements are counted and reported, not hidden.

One section at a time. The inspection skips elements that are not visible — and rightly so: a hidden element has no contrast to measure. But `index.html` alternates 89 sections and shows only one: out of **19,751** text elements the bench was looking at **41**, and printing "0 defects". It was not a false result, it was a result on a sample nobody had declared. The sections are now opened one by one, and **the number of sections traversed is printed**.

That count earned its keep on the very first run: the first attempt traversed **zero** — Playwright's click waits for the element to be visible, and the buttons sit inside collapsed navigation groups — and the bench said so ("87 sections present, none traversed") instead of printing another reassuring zero.

3.5-bis The accessibility debt, and the pact that it must not grow

Walking every section, **1,069** contrast defects surfaced. Today they are **zero** — the list of causes and remedies is in `docs/09 §4` — and unlabelled fields have fallen from 40 to **zero**. The mechanism described below remains, and matters more than the number it guards. Demanding zero *from the outset* would have left only two roads, both bad: a verification red for ever, or one loosened until it turned green again. The third road is to **declare the number** and forbid it from growing, whatever it is. That is how 1,069 became zero: every step recorded, none of them reversible in silence.

File	Role
<code>docs/evidence/accessibilita-riferimento.json</code>	The recorded count

Situation	Bench outcome
The count rises	✗ FAILED — it is a regression
The count falls	✓ passes, and asks for the baseline to be updated
The count is unchanged	✓ passes silently: debt declared, not grown — today that value is 0 , and the bench defends it

```
node tools/verifica-accessibilita.js --aggiorna-riferimento
```

To be used **only after reducing** the defects, never to turn a regression green. Even at zero this is not full WCAG conformance: text inside SVGs and text over gradients stay outside, which automation cannot judge and which the bench **counts and reports** on every run (`docs/09 D-09`). It is the honest measure of what can be measured, with the guarantee that it will not silently get worse.

3.6 Consistency

Bench	Verifies
<code>verifica_guida</code>	That the documentation's promises exist in the code; absence of conflict markers across 58 files
<code>verifica-documenti</code>	Internal links, version alignment, index consistency, justification of the deviations, internal consistency of the traceability matrix (no duplicated identifiers, coverage totals equal to the identifiers actually defined)
<code>genera-pacchetti</code>	Rewrites the references of the extracted documents and verifies that no link is left broken in the three delivery packages
<code>genera-pdf</code>	Converts the documents to PDF and fails if a file falls below the plausibility threshold (empty conversion)
<code>verifica-affermazioni</code>	Compares every number declared in the documents — sections, drugs, diseases, tumours, strategies, modules — with the one measured in the running application , in Italian <i>and</i> in English

Why numbers must be measured, not remembered. The technical dossier opens with figures: "63 diseases (23 tumours)", "46 strategies", "25 modules". They are the first thing an assessor reads, and they are enough for everything else to be believed or doubted. Nobody was checking them: written once, they stayed put while the application changed. And indeed `chimorga.html` declared **"25 modules"** at the top and **"17 modules"** further down — two statements about the same object, in the same file, one of them false.

A lesson paid for while writing the bench. I had already concluded that "63 diseases" was an exaggeration: in the source I had found a `DISEASES` array with 12 entries, and I was about to "correct" the document. Measuring in the DOM I saw that the menu really does list 63 — the array I had found was a different, smaller one. **A count on the source is not a measurement: it is a clue.** That is why the bench opens the page, opens the section, and counts.

It also checks that the documents agree **with one another**: two documents saying different things about the same object are a defect even when one of them is right. The English translation counts as one of those documents — `docs/en/05` had been left at "84 sections" while the app had 87, and had been stuck at version `bsi-v146` for three releases — and no bench saw it, because none looked at the English set.

Why a check on the matrix. The matrix declared `SCI-10` twice — once as a requirement verified at 100 %, once as an explicitly **unverified** one — and the summary table added up to 36 requirements while 37 were defined in the document. Both errors sat in the number an assessor reads first. The bench now counts the identifiers actually present and fails if the totals written do not match: the proof that the check really measures something is that, with the two errors reintroduced one at a time, it reports both.

4. Notable test cases

Some benches deserve a mention because they verify properties a functional test does not observe.

Case	Method	Measured result
Memory leaks	89 sections × 5 rounds, DOM nodes counted each round	+26,876 on the first round (construction), +0 on the four that follow
Exhausted storage	<code>setItem</code> replaced with a function that always throws	10 pages out of 10 stay operational
Degraded ≠ absent network	Requests held for 20 s, interception at context level	Answer from cache in 3,507 ms (threshold 3,500)
Spectrum reproducibility	Drawn twice, compared byte for byte	Identical
WebGL contexts	Viewer constructions over 10 consecutive molecules	From 7 to 1

Case	Method	Measured result
Corrupted history	Two Enter presses 500 ms apart	user,assistant instead of user,user,assistant,assistant
Text contrast	WCAG formula on every element with text of its own, 13 pages and 89 sections	19,751 elements examined (it was 41): 1,069 defects surfaced, 1,069 corrected , 0 recorded as a value that must not grow
Immediate cancellation	Stop pressed at 1.5 s, state sampled every second	Send button available from the 1st second (it was the 10th)

5. How to add a bench

Minimal structure

```

/* A comment explaining WHAT this bench demonstrates and why it is needed. */
const { chromium } = require('playwright-core');
const CHROME = '/opt/pw-browsers/chromium-1194/chrome-linux/chrome';

(async () => {
  const b = await chromium.launch({ executablePath: CHROME, args: ['--no-sandbox'] });
  // serviceWorkers:'block' if the bench modifies files: without it the SW
  // might serve a cached copy and the old file would be measured
  const ctx = await b.newContext({ serviceWorkers: 'block' });
  const pg = await ctx.newPage();
  const err = [];
  pg.on('pageerror', e => err.push(e.message));

  let ok = 0, ko = 0;
  const att = (d, atteso, avuto) => {
    if (String(atteso) === String(avuto)) { ok++; console.log(' ✓ ' + d + ' → ' + avuto); }
    else { ko++; console.log(' x ' + d + '\n      expected: ' + atteso + '\n      got: ' +
+ avuto); }
  };

  await pg.goto('http://127.0.0.1:8899/index.html', { waitUntil: 'load', timeout: 60000 });
  await pg.waitForTimeout(2500);

  // ... measurements ...

  att('no JS error', 0, err.length);
  await b.close();
  console.log('\n' + (ko ? 'x ' + ko + ' FAILED, ' : '') + ok + ' passed');
  process.exit(ko ? 1 : 0); // the exit code is what counts
})();

```

The name is then added to the appropriate family in `tools/genera-evidenza.js`.

The rule that matters

A bench that measures nothing passes. It is the most insidious way to obtain a false guarantee: no alarm, no failure, and no verification.

Therefore **every bench that simulates a condition must count how many times the simulation actually fired, and fail if that count is zero.**

Traps actually met while building the battery:

Trap	How it showed itself
<code>page.route</code> does not intercept Service Worker requests (<code>context.route</code> is needed)	The "degraded network" test measured a healthy network and concluded in 16 ms instead of 3,500
Probing exhausted <code>localStorage</code> with a one-byte write	After saturation one byte always finds room: the bench "passed"
Looking for <code>[data-tab]</code> where navigation uses <code>[data-s]</code>	Zero tabs found, therefore zero measurements — and zero failures
The ticks of an axis on <code><canvas></code> are not text	They must be collected by intercepting <code>fillText</code> ; and the bottom-most line is the axis <i>title</i> , not a tick
A probe installed on a library loaded afterwards	It is replaced along with the library: zero calls counted while the code was working
A <code>var</code> inside an IIFE is not <code>window.something</code>	Assigning it from outside creates a different variable

6. Coverage gaps

Declared. The coverage reported in `08-Traceability-Matrix.md` measures **how many requirements have a bench**, not how many lines of code are executed: they are two different things and must not be confused.

Gap	Current situation	Recommendation
Code coverage	Measured: 49.89 % of statements over the widest path. What stays outside is whole-battery coverage and branch coverage: an <code>if</code> entered from one side only counts as covered	Extend collection to every bench, and move from statement coverage to branch coverage
Accessibility	Automated over 13 pages and all 89 sections: WCAG contrast, accessible names, labels, alternative text, heading hierarchy. Text inside SVGs and over gradients stay outside, and are counted on every run	Add <code>axe-core</code> alongside, for the rules this bench does not implement (ARIA roles, tab order, focus management)
Security	<code>verifica-sicurezza</code> runs 10 checks over 288 tracked files and covers SEC-01, SEC-03, SEC-05, SEC-06; SEC-04 is covered by <code>verifica_guida</code> . SEC-02 remains indirect : see <code>docs/09 D-03</code>	Observe the network traffic during real use, the only direct verification of SEC-02
Browsers other than Chromium	No automatic test on Firefox or WebKit	Extend the main benches to <code>webkit</code> , where the differences on IndexedDB and Service Worker are greatest
Performance	Manual cross-device testing	Automatic measurement of first-paint time

Gap	Current situation	Recommendation
Visual regression	Absent	Screenshot comparison for the charts, which are the heart of the product

7. Acceptance criteria

A version is not published if even one of these is unsatisfied.

- ☐ `node tools/genera-evidenza.js` closes with **0 failures**
- ☐ `tools/verifica-farmaci.js`: no defect, every deviation recorded with its reason
- ☐ `verifica_guida`: no promise in the documentation without a counterpart
- ☐ No non-network JavaScript error on any page
- ☐ Evidence report regenerated against the commit being published

Document updated to version `bsi-v181`.